# SOP: Bringing Up a NEMA 17 Joint

Arctos Robotic Arm — MKS Servo42D over CAN

Applies to Joints A, B and C

September 2026

This is the step-by-step procedure for taking a NEMA 17 joint from unpowered to ready for use. It is procedure only — for protocol details, encoder maths, wiring and troubleshooting, see `arctos_motor_guide.tex`.

**Do the steps in order.** Each one gates the next: the procedure moves from information to motion, and never commands motion before the joint has been proven to answer correctly.

| Joint | CAN ID | Gear ratio | Role |
|---|---|---|---|
| A | `0x04` | 48.0:1 | Elbow |
| B | `0x05` | 67.82:1 | Wrist rotation |
| C | `0x06` | 67.82:1 | Wrist joint |

Table 1: Substitute your joint's values wherever the procedure says `<ID>` and `<RATIO>`. Examples below use Joint A.

## Step 0 — Before applying power

1. Confirm the controller is the **CAN variant** of the MKS Servo42D. Read the part number on the board. An RS485 variant has no CAN transceiver and will never respond, no matter what you do in software.
2. With everything powered off, measure `CAN_H` to `CAN_L`. Expect $\approx 60\,\Omega$.
3. Confirm only `CAN_H`, `CAN_L` and `GND` run between the CANable and the controller.
4. Confirm the joint is mechanically clear and, if it is on the arm, that you know its travel limits.

**Pass:** $\approx 60\,\Omega$ across the bus and a confirmed CAN-variant board.

**If it fails:** $120\,\Omega$ means one terminator; open means broken wiring. Fix before continuing.

## Step 1 — Configure the controller panel

Power the controller. On its display, set and **save**:

Power-cycle the controller after saving.

| | |
|---|---|
| CAN ID | `<ID>` for this joint |
| CAN Rate | `500` |
| CAN Response (CANRSP) | `ENABLE` |
| CAN CRC | `SUM-8` |
| Working current | $\leq$ 1.2 A (the NEMA 17's rating) |

**Caution: Check the working current**

Panel defaults have been found as high as 3000 mA against a 1.2 A motor. Set this explicitly.

---

**Pass:** Settings persist across the power cycle.

## Step 2 — Bring up the CAN interface

```
lsusb                                          # expect ID 16d0:117e CANable2
ls -l /dev/serial/by-id/                       # note the stable device name
sudo pkill slcand
sudo ip link delete can0 2>/dev/null
sudo slcand -o -c -s6 /dev/serial/by-id/<your-canable> can0
sudo ip link set can0 txqueuelen 1000
sudo ip link set up can0
ip -br link show can0
```

**Pass:** `can0  UP`

**If it fails:** Re-check the device name — it changes whenever the adapter re-enumerates. `bitrate 0` in `ip -details` output is normal and not a fault.

## Step 3 — Confirm the real CAN ID

Never assume the ID from a config file or a table.

```
python3 src/arctos_can_control/arctos_can_control/can_id_scan.py \
    --channel can0 --ids 1-16
```

This sends only the read-only `0x31` query. It never enables coils and never commands motion, so it is safe at any time.

**Pass:** Exactly one responder, and its ID is the one you set in Step 1.

**If it fails:** No responders: re-check panel settings, wiring, and the board's part number. Multiple responders: another driver is powered on the bus — resolve the collision before going further.

## Step 4 — Read-only health check

With the ID confirmed, verify the link is trustworthy before commanding anything.

```
python3 src/arctos_can_control/arctos_can_control/joint_reliability_test.py \
    --channel can0 --can-id <ID> --gear-ratio <RATIO> --samples 200
```

| Query | Opcode | Expected |
|---|---|---|
| Encoder | `0x31` | stable; zero jitter, zero drift |
| Velocity | `0x32` | zero |
| Error | `0x39` | small (tens of counts) |
| Enable state | `0x3A` | `01` — these drivers power up enabled |
| Shaft protection | `0x3E` | `00` |

Check the following, all with the joint at rest:

**Pass:** 100% response rate, 0 checksum failures, 0 encoder jitter at rest.

**If it fails:** Any checksum failure or dropped response is a wiring or bitrate problem. Do not proceed to motion.

## Step 5 — Check the mechanism by hand

Do this before the motor ever drives the joint. It separates mechanical faults from electrical ones, which the bus alone cannot distinguish.

1. Disable the coils: `cansend can0 <ID>#F300<crc>`.
2. Confirm with `0x3A` that the state is now `00`.
3. Turn the joint output by hand, gently, both directions, while watching the encoder.

---

**Caution: Only drop coils if the joint is not gravity-loaded**

On an assembled arm this removes holding torque and the joint may fall. Support it, or skip this step if it cannot be done safely.

---

**Pass:** The encoder tracks your hand in both directions, and the motion feels smooth. This proves the gearbox transmits and nothing is skipping.

**If it fails:** Gritty or catching resistance means the gear mesh needs lubrication — use a plastic-safe PTFE or silicone grease. No encoder movement while the output turns means the motor shaft is not following: check the coupling, grub screw, or bore.

## Step 6 — First commanded motion

Re-enable coils, then command one small move. **Have the emergency stop ready in a second terminal before you send anything:**

```
# Joint A example -- recompute the checksum for your joint
cansend can0 004#F7FB
```

```
python3 src/arctos_can_control/arctos_can_control/joint_unstick_move_test.py \
    --can-id <ID> --gear-ratio <RATIO> \
    --step-degrees 0.25 --num-steps 1 --max-unstick 0 \
    --band-lo -1 --band-hi 1
```

**Caution: Set the band explicitly**

Some joints have `home_position` and `opposite_limit` of `0.0` in `arctos_controller.yaml`, which produces an inverted, unusable safe band. Always pass `--band-lo`/`--band-hi` yourself.

**Pass:** The step reports ≈100% of commanded and the status frames show `FD=[1, 2]`. Both frames matter: `FD 01` alone means the command started but did not finish.

**If it fails:** Do not retry blindly — that only heats the motor. Go back to Step 5.

## Step 7 — Stepped travel test

Extend to real travel, one direction then the other. Use a step size and count that stay inside the joint's safe range.

```
# forward
python3 src/arctos_can_control/arctos_can_control/joint_unstick_move_test.py \
    --can-id <ID> --gear-ratio <RATIO> \
    --step-degrees 2.0 --num-steps 9 --max-unstick 0 \
    --band-lo -25 --band-hi 25 --speed 25

# reverse -- same step count, to close the loop
python3 src/arctos_can_control/arctos_can_control/joint_unstick_move_test.py \
    --can-id <ID> --gear-ratio <RATIO> \
    --step-degrees -2.0 --num-steps 9 --max-unstick 0 \
    --band-lo -25 --band-hi 25 --speed 25
```

**Pass:** Every step 99–101% of commanded with `FD 02`, and the round trip returns to the starting position. On a healthy joint this closes to within a few encoder counts.

**If it fails:** A step that under-delivers and then a step that delivers nothing means the joint is against something, or the motor has lost drive. Stop and diagnose — do not increase torque or retry.

## Step 8 — Acceptance

The joint is ready for use when all of the following hold:

| Check | Criterion |
|---|---|
| CAN ID | confirmed by scan, matches config |
| Comms | 100% response, 0 checksum failures |
| Encoder at rest | 0 jitter, 0 drift |
| Hand check | smooth, encoder tracks both directions |
| Per-step accuracy | 99–101% of commanded |
| Status frames | `FD 02` on every step |
| Round trip | returns to start within a few counts |
| Shaft protection | still `00` |

## Step 9 — Operating limits

Once accepted, keep within these when using the joint:

- **Speed at or below 225** (motor rpm). Above ≈250 the motor receives more pulses than it executes and silently loses position. It saturates near 310 rpm.
- **Use `0xFD` for all motion.** Do not use `0xF5`.
- **Never retry into a stall.** Back off and re-attempt, or stop.
- **Disable coils between tests** so the motor is not needlessly energized — unless the joint is gravity-loaded, in which case leave it holding.

## Step 10 — Shutdown

```
# stop any running script first (Ctrl+C)
cansend can0 <ID>#F300<crc>     # disable coils, if safe to do so
sudo ip link set can0 down
sudo pkill slcand
# then unplug the adapter
```